Desafio Final — Pipeline de pagamentos (entrega de exemplo)

> **Isto é um esqueleto de entrega nota-alta**, usado no vídeo-gabarito e como template. As seções marcadas `<sua evidência aqui>` são o que **você** captura do seu pipeline; as preenchidas mostram o formato esperado.

Pipeline: Neon (Postgres) → CDC Debezium → tópicos Avro → Flink SQL (enriquecimento + fraude) → consumo. Reproduzível com `./setup.sh` (pré-requisito: `.env` preenchido a partir do `.env.example`).

Camada 1 — Fundação

- Environment `desafio-final` + cluster Basic `desafio-basic` (aws/us-east-1)
- Identidades: `desafio-producer` (WRITE em `desafio-*`), `desafio-consumer` (READ em `desafio-*` + grupo `desafio-*`); conta pessoal só em setup/teardown

```
<sua evidência aqui: confluent iam service-account list + acl list por SA>
```

Camada 2 — Contrato de dados

Schemas em `schemas/`, namespace `com.bootcamp.payments.*`, money como `bytes` + `decimal(15,2)`, tags de PII/PCI nos campos sensíveis.

Evolução BACKWARD demonstrada nos dois sentidos:

```
<sua evidência aqui: test-compatibility aceitando campo novo com default>
<sua evidência aqui: test-compatibility REJEITANDO remoção de campo obrigatório>
```

Camada 3 — Ingestão (CDC)

Conector `PostgresCdcSourceV2` com `cdc_user` (menor privilégio) e `heartbeat.interval.ms=60000` — config em `connectors/cdc.json`, sem secrets (env vars).

Seed determinístico carregado (`seed=42`):

```
 customers | accounts | cards | merchants | transactions
-----------+----------+-------+-----------+--------------
       200 |      240 |   320 |        60 |         2000
```

Os três tipos de evento capturados — envelopes completos em `evidencias/01-envelopes-cdc.json`:

- `op=c` (INSERT): `before=null`, `after` com a linha nova
- `op=u` (UPDATE): `before` **preenchido** (`REPLICA IDENTITY FULL`) + `after`
- `op=d` (DELETE): `after=null` + tombstone na sequência

Camada 4 — Processamento (Flink SQL)

Statements em `sql/`: temporal join transações ⋈ contas (PK nos dois lados, `sql.tables.scan.idle-timeout=5s`) e `RULE_VELOCITY` via `MATCH_RECOGNIZE` (`PATTERN (T{3,}?) WITHIN INTERVAL '60' SECOND`, `PARTITION BY card_id`).

```
<sua evidência aqui: o alerta emitido no tópico desafio.fraud.detected
 + as 3 transações (mesmo card_id, <60s) que o dispararam>
```

Camada 5 — Operação: as quatro respostas

Pilar	Resposta (medida)
① Observabilidade	`<pico de received_bytes e lag por grupo — curl da Metrics API>`
② Segurança	`<tabela: SA → ACLs → porquê>`
③ Custos	`<billing cost list do período + 2 alavancas>`
④ Confiabilidade	`<tabela trecho a trecho; elo fraco: consumer → efeito (idempotência é código meu)>`

Custo total e teardown

- Custo declarado: `<US$ X,XX — billing cost list>`
- `./teardown.sh` executado ao final — verificação de listas vazias no output:

```
<sua evidência aqui: connect cluster list / flink statement list → None found.>
```